```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print("TF version:", tf.__version__)
```

```
TF version: 2.19.0
```

```
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()

print("x_train shape:", x_train.shape)
print("y_train shape:", y_train.shape)
print("x_test shape:", x_test.shape)
print("unique labels in train:", np.unique(y_train))
```

```
x_train shape: (60000, 28, 28)
y_train shape: (60000,)
x_test shape: (10000, 28, 28)
unique labels in train: [0 1 2 3 4 5 6 7 8 9]
```

```
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

val_frac = 0.1
val_n = int(len(x_train) * val_frac)
x_val = x_train[:val_n]
y_val = y_train[:val_n]
x_train_small = x_train[val_n:]
y_train_small = y_train[val_n:]

x_train_cnn = np.expand_dims(x_train_small, -1)
x_val_cnn   = np.expand_dims(x_val, -1)
x_test_cnn  = np.expand_dims(x_test, -1)

print("After split: train:", x_train_small.shape, "val:", x_val.shape, "test:", x_test.shape)
print("CNN shapes:", x_train_cnn.shape, x_val_cnn.shape, x_test_cnn.shape)
```

```
After split: train: (54000, 28, 28) val: (6000, 28, 28) test: (10000, 28, 28)
CNN shapes: (54000, 28, 28, 1) (6000, 28, 28, 1) (10000, 28, 28, 1)
```

```
plt.figure(figsize=(8,3))
for i in range(15):
    ax = plt.subplot(1,15,i+1)
    ax.imshow(x_train_small[i], cmap='gray')
    ax.axis('off')
plt.suptitle('Sample images (first 15 of training set)')
plt.show()
```

Sample images (first 15 of training set)

```
def build_ann():
    model = keras.Sequential([
        layers.Flatten(input_shape=(28,28)),
        layers.Dense(128, activation='relu'),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

ann = build_ann()
ann.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

```
ann.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `in
  super().__init__(**kwargs)
```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
flatten_6 (Flatten)	(None, 784)	0
dense_13 (Dense)	(None, 128)	100,480
dense_14 (Dense)	(None, 64)	8,256
dense_15 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

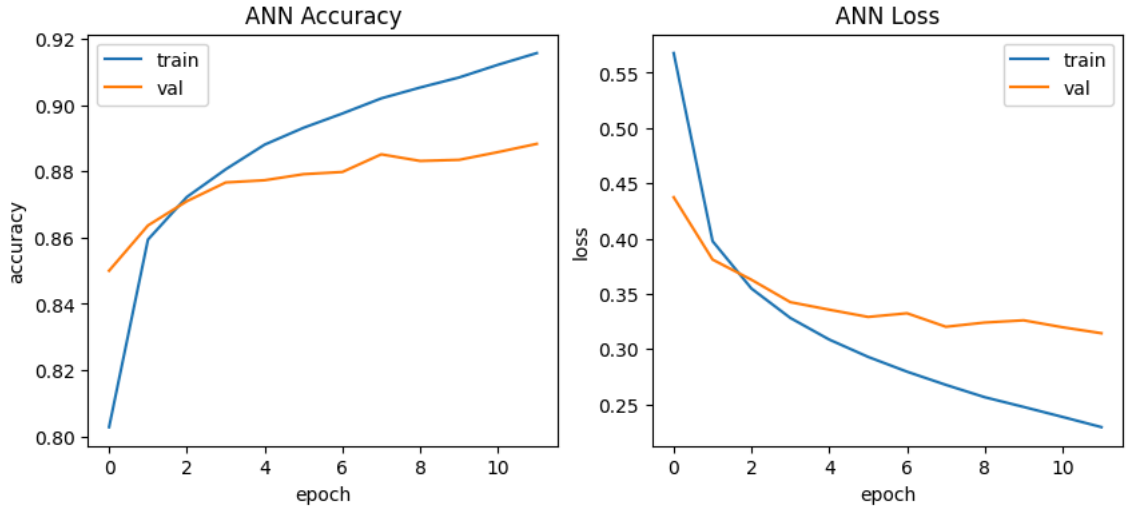
```
history_ann = ann.fit(x_train_small, y_train_small,
                      epochs=12,
                      batch_size=128,
                      validation_data=(x_val, y_val),
                      verbose=2)

loss_ann, acc_ann = ann.evaluate(x_test, y_test, verbose=0)
print(f"ANN test loss: {loss_ann:.4f}   test acc: {acc_ann:.4f}")
```

```
Epoch 1/12
422/422 - 4s - 9ms/step - accuracy: 0.8027 - loss: 0.5678 - val_accuracy: 0.8500 - val_loss: 0.4373
Epoch 2/12
422/422 - 1s - 2ms/step - accuracy: 0.8594 - loss: 0.3976 - val_accuracy: 0.8637 - val_loss: 0.3809
Epoch 3/12
422/422 - 1s - 2ms/step - accuracy: 0.8723 - loss: 0.3547 - val_accuracy: 0.8710 - val_loss: 0.3627
Epoch 4/12
422/422 - 1s - 2ms/step - accuracy: 0.8806 - loss: 0.3281 - val_accuracy: 0.8767 - val_loss: 0.3424
Epoch 5/12
422/422 - 1s - 3ms/step - accuracy: 0.8881 - loss: 0.3085 - val_accuracy: 0.8773 - val_loss: 0.3356
Epoch 6/12
422/422 - 1s - 3ms/step - accuracy: 0.8932 - loss: 0.2928 - val_accuracy: 0.8792 - val_loss: 0.3290
Epoch 7/12
422/422 - 1s - 2ms/step - accuracy: 0.8975 - loss: 0.2795 - val_accuracy: 0.8798 - val_loss: 0.3323
Epoch 8/12
422/422 - 1s - 2ms/step - accuracy: 0.9021 - loss: 0.2675 - val_accuracy: 0.8852 - val_loss: 0.3201
Epoch 9/12
422/422 - 1s - 3ms/step - accuracy: 0.9054 - loss: 0.2563 - val_accuracy: 0.8832 - val_loss: 0.3239
Epoch 10/12
422/422 - 1s - 2ms/step - accuracy: 0.9084 - loss: 0.2475 - val_accuracy: 0.8835 - val_loss: 0.3259
Epoch 11/12
422/422 - 2s - 4ms/step - accuracy: 0.9122 - loss: 0.2385 - val_accuracy: 0.8858 - val_loss: 0.3197
Epoch 12/12
422/422 - 2s - 4ms/step - accuracy: 0.9158 - loss: 0.2293 - val_accuracy: 0.8883 - val_loss: 0.3143
ANN test loss: 0.3491   test acc: 0.8805
```

```
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history_ann.history['accuracy'], label='train')
plt.plot(history_ann.history['val_accuracy'], label='val')
plt.title('ANN Accuracy')
plt.xlabel('epoch'); plt.ylabel('accuracy'); plt.legend()

plt.subplot(1,2,2)
plt.plot(history_ann.history['loss'], label='train')
plt.plot(history_ann.history['val_loss'], label='val')
plt.title('ANN Loss')
plt.xlabel('epoch'); plt.ylabel('loss'); plt.legend()
plt.show()
```




```
def build_cnn():
    model = keras.Sequential([
        layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        layers.MaxPooling2D((2,2)),
        layers.Conv2D(64, (3,3), activation='relu'),
        layers.MaxPooling2D((2,2)),
        layers.Flatten(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

cnn = build_cnn()
cnn.compile(optimizer=keras.optimizers.Adam(learning_rate=1e-3),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
cnn.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_10 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_11 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_11 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_7 (Flatten)	(None, 1600)	0
dense_16 (Dense)	(None, 64)	102,464
dense_17 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)

```
history_cnn = cnn.fit(x_train_cnn, y_train_small,
                      epochs=12,
                      batch_size=128,
                      validation_data=(x_val_cnn, y_val),
                      verbose=2)

loss_cnn, acc_cnn = cnn.evaluate(x_test_cnn, y_test, verbose=0)
print(f"CNN test loss: {loss_cnn:.4f}   test acc: {acc_cnn:.4f}")
```

```
Epoch 1/12
422/422 - 6s - 14ms/step - accuracy: 0.7884 - loss: 0.5874 - val_accuracy: 0.8578 - val_loss: 0.4055
Epoch 2/12
422/422 - 2s - 4ms/step - accuracy: 0.8690 - loss: 0.3717 - val_accuracy: 0.8798 - val_loss: 0.3468
Epoch 3/12
422/422 - 2s - 4ms/step - accuracy: 0.8837 - loss: 0.3245 - val_accuracy: 0.8873 - val_loss: 0.3220
Epoch 4/12
422/422 - 2s - 4ms/step - accuracy: 0.8947 - loss: 0.2940 - val_accuracy: 0.8883 - val_loss: 0.3033
```

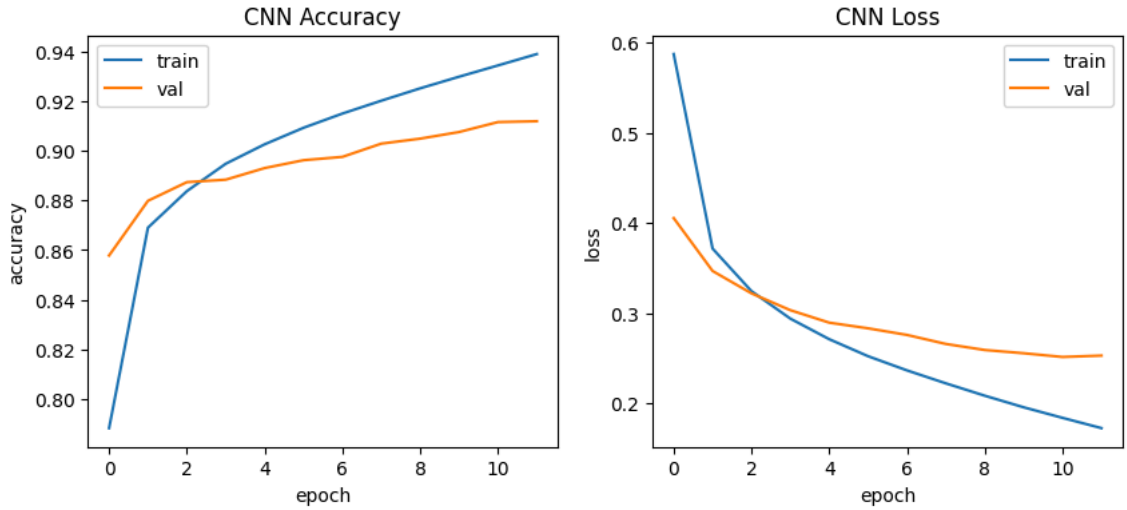
```
Epoch 5/12
422/422 - 3s - 7ms/step - accuracy: 0.9026 - loss: 0.2712 - val_accuracy: 0.8930 - val_loss: 0.2895
Epoch 6/12
422/422 - 2s - 4ms/step - accuracy: 0.9092 - loss: 0.2523 - val_accuracy: 0.8962 - val_loss: 0.2832
Epoch 7/12
422/422 - 2s - 4ms/step - accuracy: 0.9149 - loss: 0.2366 - val_accuracy: 0.8975 - val_loss: 0.2759
Epoch 8/12
422/422 - 2s - 4ms/step - accuracy: 0.9201 - loss: 0.2222 - val_accuracy: 0.9028 - val_loss: 0.2659
Epoch 9/12
422/422 - 2s - 4ms/step - accuracy: 0.9251 - loss: 0.2085 - val_accuracy: 0.9048 - val_loss: 0.2592
Epoch 10/12
422/422 - 2s - 4ms/step - accuracy: 0.9297 - loss: 0.1956 - val_accuracy: 0.9075 - val_loss: 0.2556
Epoch 11/12
422/422 - 2s - 5ms/step - accuracy: 0.9343 - loss: 0.1839 - val_accuracy: 0.9115 - val_loss: 0.2515
Epoch 12/12
422/422 - 2s - 5ms/step - accuracy: 0.9389 - loss: 0.1725 - val_accuracy: 0.9118 - val_loss: 0.2530
CNN test loss: 0.2730   test acc: 0.9068
```

cnn

```
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history_cnn.history['accuracy'], label='train')
plt.plot(history_cnn.history['val_accuracy'], label='val')
plt.title('CNN Accuracy'); plt.xlabel('epoch'); plt.ylabel('accuracy'); plt.legend()

plt.subplot(1,2,2)
plt.plot(history_cnn.history['loss'], label='train')
plt.plot(history_cnn.history['val_loss'], label='val')
plt.title('CNN Loss'); plt.xlabel('epoch'); plt.ylabel('loss'); plt.legend()
plt.show()

print("Final test accuracy — ANN: {:.4f}   CNN: {:.4f}".format(acc_ann, acc_cnn))
```




```
Final test accuracy — ANN: 0.8805   CNN: 0.9068
```

```
grid = [{'lr':1e-3,'batch':128}, {'lr':5e-4,'batch':128},
        {'lr':1e-3,'batch':64},  {'lr':5e-4,'batch':64}]
results = []
for cfg in grid:
    print("Testing:", cfg)
    m = build_cnn()
    m.compile(optimizer=keras.optimizers.Adam(learning_rate=cfg['lr']),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
    h = m.fit(x_train_cnn, y_train_small, epochs=3, batch_size=cfg['batch'],
              validation_data=(x_val_cnn, y_val), verbose=0)
    loss_t, acc_t = m.evaluate(x_test_cnn, y_test, verbose=0)
    results.append({'cfg':cfg, 'test_loss':loss_t, 'test_acc':acc_t})

# result
import pandas as pd
pd.DataFrame([{'lr':r['cfg']['lr'],'batch':r['cfg']['batch'],'test_acc':r['test_acc']} for r in results])
```

```
Testing: {'lr': 0.001, 'batch': 128}
Testing: {'lr': 0.0005, 'batch': 128}
Testing: {'lr': 0.001, 'batch': 64}
Testing: {'lr': 0.0005, 'batch': 64}
```

	lr	batch	test_acc
0	0.0010	128	0.8802
1	0.0005	128	0.8650

prediction using cnn

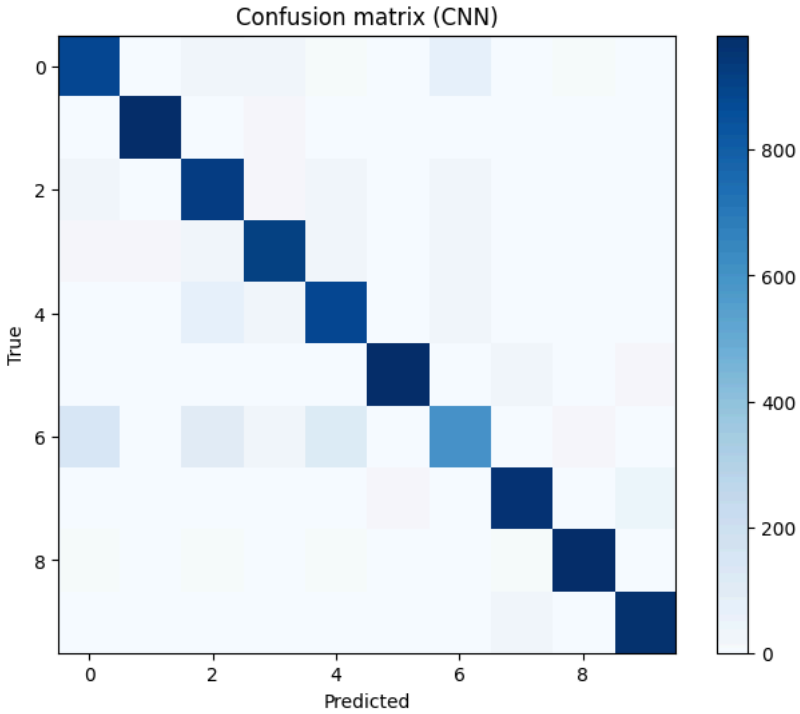
```
from sklearn.metrics import confusion_matrix, classification_report
y_pred = np.argmax(cnn.predict(x_test_cnn), axis=1)
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
plt.title("Confusion matrix (CNN)")
plt.colorbar()
plt.xlabel('Predicted'); plt.ylabel('True')
plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
              precision    recall  f1-score   support

           0       0.83      0.89      0.86      1000
           1       0.98      0.98      0.98      1000
           2       0.81      0.92      0.87      1000
           3       0.92      0.91      0.91      1000
           4       0.82      0.88      0.85      1000
           5       0.98      0.98      0.98      1000
           6       0.83      0.60      0.69      1000
           7       0.96      0.96      0.96      1000
           8       0.97      0.98      0.98      1000
           9       0.96      0.97      0.97      1000

    accuracy                           0.91     10000
   macro avg       0.91      0.91      0.90     10000
weighted avg       0.91      0.91      0.90     10000
```




```
ann.save('ann_fashion_mnist.h5')
cnn.save('cnn_fashion_mnist.h5')
print("Saved models to disk.")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. Th
Saved models to disk.
```